Contents

1 Introduction **3**

2 Installation **4**
2.1 Requirements . . . 4
2.2 Installing . . . 4

3 Initialization **4**

4 Concepts **5**
4.1 Creating widgets . . . 5
4.2 Configuring widgets . . . 5
4.3 Size . . . 5
4.4 State . . . 5
4.5 Variables . . . 6
4.6 Colors . . . 6
4.6.1 Named color options . . . 6
4.6.2 The theme dictionary . . . 7
4.7 Fonts . . . 7
4.8 Images . . . 7
4.9 Events . . . 7
4.10 Native widgets inside . . . 8

5 Widgets **8**
5.1 Button . . . 9
5.2 Label . . . 10
5.3 Checkbutton . . . 10
5.4 Radiobutton . . . 11
5.5 Toggle . . . 11
5.6 Entry . . . 12
5.7 Textbox . . . 13
5.8 Spinbox . . . 14
5.9 Combobox . . . 14
5.10 OptionMenu . . . 15
5.11 Menu . . . 16
5.12 MenuBar . . . 16
5.13 Scale . . . 17
5.14 Progressbar . . . 17
5.15 Meter . . . 18
5.16 ColorDisplay . . . 18
5.17 Scrollbar . . . 19
5.18 Separator . . . 19
5.19 Listbox . . . 19
5.20 Notebook . . . 20
5.21 Panedwindow . . . 22

5.22 GridPanedwindow . . . 23
5.23 Frame . . . 25
5.24 ScrolledFrame . . . 26
5.25 Foldable . . . 26
5.26 LabelFrame . . . 27
5.27 Tooltip . . . 28

6 Windows **28**
6.1 Window . . . 28
6.2 Toplevel . . . 29

7 File dialogs **29**
7.1 Functions . . . 29
7.2 Options . . . 30
7.3 Using the dialog . . . 31
7.4 Helper functions . . . 32

8 Message boxes **32**

9 Color picker **33**
9.1 Using the dialog . . . 33
9.2 Helper functions . . . 33

10 Appearance and themes **34**
10.1 Appearance modes . . . 34
10.2 Accent color . . . 34
10.3 Theme keys . . . 35
10.4 Changing the theme dynamically . . . 36

11 Functions **38**
11.1 Set_appearance_mode, Get_appearance_mode . . . 39
11.2 Set_theme, Get_theme, Reset_theme . . . 39
11.3 Expand_theme . . . 39
11.4 Hover_variant . . . 39
11.5 Window_theme . . . 39
11.6 Load_canvas_image . . . 39
11.7 Make_font . . . 39
11.8 Resolve_color . . . 40
11.9 Parent_background . . . 40
11.10 Rounded_box_image . . . 40

12 Writing your own widget **40**

13 Notes and limitations **41**

CanvasTK Docs

Rafugafu

Note: You can always combine canvastk with normal tkinter. Every canvastk widget is a normal tkinter widget (a `tkinter.Canvas` or a `tkinter.Frame`), so `grid`, `pack`, `place`, `bind`, `after`, `configure`, `cget`, and so on all work.
Note: These are the CanvasTK docs, not the tkinter docs. Anything of tkinter will not be explained. You can find the tkinter docs here.
Note: Most of the examples are not full code. They usually need to be preceded by `import canvastk as ctk ; root = ctk.Window()` and succeeded by `root.mainloop()`.

1 Introduction

CanvasTK is a widget toolkit for tkinter in which the widgets are drawn on a canvas instead of by the operating system. Every rounded box, border, track, thumb, tab, and popup is an anti-aliased (4x supersampled) image that is cached and shared between all widgets, so the widgets look the same everywhere, can be rounded, and can be recolored completely.
Features:

- Light and dark appearance modes and any accent color (with five ready-made ones), all switchable while the program is running.
- Every color, and also fonts, radii, border widths, and sizes, can be customized per widget.
- Large widgets are drawn from cached corner pieces, so dragging panes and resizing windows stays smooth.
- Widgets that need real text editing (Entry, Textbox, Spinbox, and Combobox) embed a native `tkinter.Entry` or `tkinter.Text` on top of canvas chrome, so cursor, selection, clipboard, undo, and tags all behave exactly like tkinter.

2 Installation

2.1 Requirements

You need Python 3 with tkinter, and the Pillow library (`pip install pillow`). Nothing else.

2.2 Installing

The library is a folder (a package) called `canvastk`. To use it in one application, copy the whole `canvastk` folder next to your program. To make it available to all your programs, copy the folder into your Python libraries folder (`python3.*/site-packages`).
After that, `import canvastk` works. The version is available as `canvastk.__version__`.

3 Initialization

To make a window, use

```
import canvastk as ctk
root = ctk.Window()
root.mainloop()
```

`Window` is a `tkinter.Tk` whose background follows the appearance mode. If you want a subwindow of an already existing window, use `ctk.Toplevel(root)` (see Windows).
Here is a complete small program:

```
import canvastk as ctk

ctk.set_appearance_mode('dark')
ctk.set_theme(ctk.CANVAS_COLOR_THEMES['green'])

root = ctk.Window()
root.title('Hello')
ctk.Label(root, 'Your name:').grid(row = 0, column = 0, padx = 10, pady = 10)
name = ctk.Entry(root)
name.grid(row = 0, column = 1, padx = 10, pady = 10)
ctk.Button(root, 'Greet',
command = lambda: print('Hello', name.get())).grid(
row = 1, column = 0, columnspan = 2, pady = 10)
root.mainloop()
```

The file `ctk_test.py` in the project folder is a demo window showing every widget.

4 Concepts

4.1 Creating widgets

Widgets are created from the module and the *first argument is always the master*:

```
b = ctk.Button(root, 'Click Me!')
b.grid()
```

The master can be a `Window`, a `Toplevel`, a Frame, or any other container, including normal tkinter ones. The second argument is usually the text (or the main value) of the widget.

4.2 Configuring widgets

Every option listed for a widget can be given when creating it, changed later with `w.configure(option = value)` (or `w.config(...)`, or `w['option'] = value`), and read with `w.cget('option')` (or `w['option']`).
Example:

```
b = ctk.Button(root, 'Click Me!',
command = lambda: b.config(state = 'disabled', text = 'Done'))
b.grid()
```

Widgets redraw themselves automatically when an option changes, and the widgets that size themselves to their content (buttons, labels, check boxes, drop-downs...) resize themselves when their text, font, or image changes.

4.3 Size

Widgets that size to their content (Button, Label, Checkbutton, Radiobutton, OptionMenu) pick their own size. You can fix it by passing `width` and/or `height` in pixels. Every canvas widget, like every tkinter widget, also stretches normally when you use `sticky`, `fill`, or `expand`.
The widgets that are measured in characters and lines like in tkinter (Entry, Textbox, Spinbox, Combobox, Listbox) use `width` in characters and `height` in lines.

4.4 State

Most widgets take `state = 'normal'` or `state = 'disabled'`. Disabled widgets use their `disabled_*` colors and ignore the mouse and keyboard. Entry and Combobox also accept `'readonly'`.

4.5 Variables

Widgets that show or edit a value take normal tkinter variables (`tkinter.StringVar`, `IntVar`, `BooleanVar`, `DoubleVar`) and update themselves when the variable changes: the `textvariable` of a label, entry, or spinbox, the `variable` of a check box, radio button, scale, progress bar, and drop-down.

4.6 Colors

4.6.1 Named color options

Every color a widget draws is a named *part*. You can override any of them for one widget, either when creating it or later with `configure`. Passing `None` goes back to the theme color.
Example:

```
e = ctk.Entry(root, border_color = '#2b8a3e', focus_color = '#51cf66',
fill_color = '#f4fff6', text_color = '#1b4332')
e.grid()
e.configure(fill_color = None)   # back to the theme
```

The color options of every widget are listed in its own section. Colors can be anything tkinter understands (`'red'`, `'#ff0000'`, `'#f00'`). A color is also remembered across appearance mode changes: an override stays, everything else follows the theme.
Hover colors. If you give a widget a fill color (for example `fill_color`, `unchecked_color`, `tab_color`, or `thumb_color`) but not the matching hover color (`hover_fill_color`, `hover_color`, `tab_hover_color`, or `thumb_hover_color`), the hover color is derived from your fill color with hover_variant, so the hover is always visible against it, exactly as for the Button's `default_bg`. This works on the OptionMenu, Combobox list, Listbox, Checkbutton, Radiobutton, Scale, Scrollbar, Notebook, Foldable, and Menu. A hover color you give always wins.
Names that are shared between many widgets mean the same thing everywhere:

- `fill_color`, `hover_fill_color`, `disabled_fill_color`: the background of the widget in its normal, hovered, and disabled states.
- `border_color`, `hover_border_color`, `disabled_border_color`: the border.
- `focus_color`: the border/ring color while the widget has the keyboard focus.
- `text_color`, `disabled_text_color`: the text. The widgets with text also accept `fg` (the same as `text_color`).
- `arrow_color`, `disabled_arrow_color`: arrows and chevrons.
- `track_color`, `thumb_color`, `progress_color`: sliders, scroll bars, and progress bars.

4.6.2 The theme dictionary

Every part falls back to a theme color (see Theme keys). Instead of the named options, you can also give a `theme` dictionary of theme keys to override for that one widget. An `'accent'` in it also derives the hover color and the other keys that follow from the accent (unless you give them), just like in the global theme:

```
ctk.Button(root, 'Red accent', theme = {'accent': '#e03131'}).grid()
```

4.7 Fonts

Every widget that shows text takes `font`. It can be a `tkinter.font.Font` object, a font tuple, a font name string, or `None` for the default font:

```
import tkinter.font as tkfont
big = tkfont.Font(family = 'Times', size = 16, slant = 'italic', weight = 'bold')
ctk.Button(root, 'Font object', font = big).grid()
ctk.Button(root, 'Font tuple', font = ('Courier', 11, 'bold')).grid()
ctk.Label(root, 'Default font').grid()
```

`font` can be changed later with `configure`, and the widget resizes to fit. The OptionMenu and the Combobox also take `popup_font` for their drop-down lists, the Menu and the Tooltip take `font`, and the Notebook takes `font` for its tab labels.
Note: Fonts made from a tuple, a name, or the default are shared between widgets (making a Tk font is slow when you make hundreds of widgets). Pass your own `Font` object if you want a font you can change later; it is used as it is.

4.8 Images

The `image` option of a button or label takes a `PhotoImage`. Use `ctk.load_canvas_image(path, size)` to load a file (any format Pillow can read, with transparency) resized to `(width, height)`:

```
icon = ctk.load_canvas_image('save.png', (16, 16))
ctk.Button(root, 'Save', image = icon).grid()
```

Keep a reference to the image (a variable) like with any tkinter image.

4.9 Events

Besides the normal tkinter events, some widgets generate virtual events:

- `<<ComboboxSelected>>` — Combobox, when a value is picked from the list.
- `<<ListboxSelect>>` — Listbox, when the user changes the selection.
- `<<NotebookTabChanged>>` — Notebook, when the selected tab changes.

- `<<NotebookTabClosed>>` — Notebook, after a tab was closed.
- `<<NotebookTabMoved>>` — Notebook, after the user dragged a tab to a new position.
- `<<PanedwindowPaneMoved>>` — Panedwindow and GridPanedwindow, after the user dragged a pane to a new place.

4.10 Native widgets inside

Entry, Textbox, Spinbox, and Combobox are frames that contain a real tkinter widget (the *inner* widget, `w.inner`). The frame forwards everything it does not have to the inner widget. So `e.get()`, `tb.insert('end', 'text')`, `tb.tag_configure(...)`, `tb.yview()`, `tb.see('end')`, and so on work directly on the canvastk widget. `w.bind(...)` binds on the inner widget (except for the frame-level events `<Configure>`, `<Map>`, `<Unmap>`, `<Destroy>`, `<Enter>`, `<Leave>`, and `<Visibility>`, which bind on the frame), and `w.focus_set()` focuses the inner widget.
You can also use `w.inner` directly when you need the exact tkinter widget.

5 Widgets

These are all the widgets:

- Button
- Label
- Checkbutton
- Radiobutton
- Toggle
- Entry
- Textbox
- Spinbox
- Combobox
- OptionMenu
- Menu
- MenuBar
- Scale
- Progressbar

- Meter
- ColorDisplay
- Scrollbar
- Separator
- Listbox
- Notebook
- Panedwindow
- GridPanedwindow
- Frame
- ScrolledFrame
- Foldable
- LabelFrame
- Tooltip

Now here are specifications and examples for each one:

5.1 Button

Use `b = ctk.Button(master, text, command, {args})` to make a button.
Example:

```
b = ctk.Button(root, 'Click Me!', command = lambda: print('clicked'))
b.grid()
```

The command is called when the mouse button is released over the button, or when the button has the focus and you press `Return` or `space`. While pressed, the button shrinks slightly (`press_shrink`).
Args (all of them can also be changed later with `configure`):

- `text`, `command`, `image`, `font` (default: bold, size 12), `state`.
- `default_bg`, `hover_bg`, `press_bg`: the background in the normal, hovered, and pressed states.
- `default_fg`, `hover_fg`, `press_fg`: the text color in those states.
- `disabled_bg`, `disabled_fg`: the colors when disabled.
- `border_width` (the outline width in pixels, default `1`) and `border_color`.

- `radius` (the corner radius; the default is the theme key `button_radius`, 15; a large radius makes a pill), `press_shrink` (default 2), `padx` (default 13), and `pady` (default 8), the space around the content.
- `focus_color`: the border color while the button has the keyboard focus, `disabled_border_color`.

Anything left as `None` follows the theme (the accent color). If you give `default_bg` but not `hover_bg`, the hover color is derived from `default_bg`: dark and medium colors get lighter on hover (a black button becomes grey), and very light colors get darker (a near-white button becomes grey), so the change is always visible (see Hover_variant). Give `hover_bg` yourself to choose it. The pressed color defaults to `default_bg` (the button just shrinks).
Example:

```
ctk.Button(root, 'cancel', default_bg = '#E9E2E2', hover_bg = '#F8F4F4',
press_bg = '#E9E2E2').grid(row = 0, column = 0)
ctk.Button(root, 'never', default_bg = '#FF0000', hover_bg = '#F04141',
press_bg = '#FF0000', default_fg = 'white', hover_fg = 'white',
press_fg = 'white').grid(row = 0, column = 1)
```

Method: `b.invoke()` runs the command as if the button was clicked (it briefly shows the pressed look).

5.2 Label

Use `l = ctk.Label(master, text, {args})` to write some text. The label is transparent (it shows the background of its master).
Example:

```
ctk.Label(root, 'Hello!').grid()
```

Args: `text`, `textvariable` (a `StringVar` whose value is shown), `image`, `font`, `anchor` (default `'w'`), `justify`, `wraplength` (in pixels, 0 for no wrapping), `padx`, `pady` (default 2), and the colors `text_color` (or `fg`) and `disabled_text_color`.
Example:

```
s = tkinter.StringVar(value = 'Loading...')
ctk.Label(root, textvariable = s, font = ('TkDefaultFont', 14)).grid()
s.set('Done!')
```

5.3 Checkbutton

Use `c = ctk.Checkbutton(master, text, variable, onvalue, offvalue, command, {args})` to make a check box.
Example:

```
v = tkinter.IntVar(value = 1)
ctk.Checkbutton(root, 'Enable feature', variable = v,
command = lambda: print(v.get())).grid()
```

If you do not give a variable one is created. The default values are `1` (checked) and `0`. A `BooleanVar` works too.
Methods: `c.select()`, `c.deselect()`, `c.toggle()`, and `c.invoke()` (toggles and calls the command). The space bar toggles it when it has the focus.
Args: `text`, `variable`, `onvalue`, `offvalue`, `command`, `font`, `state`, and:

- `indicator_size`: the size of the box in pixels (default `18`), `indicator_radius`: its corner radius (the default is the theme key `checkbox_radius`), `spacing`: the space between the box and the text (default `10`).
- Colors: `fill_color` (the checked box), `unchecked_color`, `hover_color` (the unchecked box when hovered), `disabled_fill_color`, `disabled_unchecked_color`, `border_color`, `hover_border_color`, `focus_color`, `disabled_border_color`, `check_color` (the check mark), `disabled_check_color`, `text_color`, `disabled_text_color`.

5.4 Radiobutton

Use `r = ctk.Radiobutton(master, text, variable, value, command, {args})` to make a radio button. The radio buttons that share a variable are a group.
Example:

```
choice = tkinter.StringVar(value = 'Red')
for name in ('Red', 'Green', 'Blue'):
    ctk.Radiobutton(root, name, variable = choice).grid(sticky = 'w')
```

The `value` defaults to the text. The args and the colors are the same as the Checkbutton's; here `fill_color` is the color of the dot.

5.5 Toggle

Use `t = ctk.Toggle(master, text, variable, onvalue, offvalue, command, {args})` to make an on/off switch: a rounded track with a knob that slides across. It works like a Checkbutton, with the same variable, `onvalue` and `offvalue`, `command`, methods, and arguments, but it looks like a switch. The knob slides and the track changes color smoothly, also when the state is changed by something else setting the variable.
Example:

```
v = tkinter.IntVar(value = 1)
ctk.Toggle(root, 'Notifications', variable = v,
command = lambda: print(v.get())).grid()
```

Methods: `t.select()`, `t.deselect()`, `t.toggle()`, and `t.invoke()`. The space bar toggles it when it has the focus.
Args: the same as the Checkbutton's; here `indicator_size` is the height of the track (default `22`, and the track is 1.8 times as wide). Colors: `fill_color`

(the track when on) and `unchecked_color` (the track when off), `knob_color` (the knob when off), `check_color` (the knob when on), `border_color`, `hover_border_color`, `focus_color`, and the `disabled_*` versions, besides `text_color` and `disabled_text_color`.

5.6 Entry

Use `e = ctk.Entry(master, {args})` to make a one-line text entry with a rounded border.
Example:

```
e = ctk.Entry(root, width = 30)
e.insert(0, 'Hello')
e.grid()
print(e.get())
```

Args: `placeholder`, `textvariable`, `show` (for example `'*'` for passwords), `state` (`'normal'`, `'readonly'`, `'disabled'`), `font`, `width` (in characters, default 20), `justify`, `radius` (the default is the theme key `radius`, 8), `border_width` (default 1), `focus_border_width` (default 2), and every other option of `tkinter.Entry` (for example `validate` and `validatecommand`), which is passed to the inner entry.
Colors: `fill_color`, `disabled_fill_color`, `border_color`, `focus_color`, `disabled_border_color`, `text_color`, `disabled_text_color`, `cursor_color` (the text cursor), `selection_color`, `selection_text_color`. The tkinter names `bg`, `fg`, `insertbackground`, `selectbackground`, `selectforeground`, `disabledbackground`, and `disabledforeground` are accepted and mean the same.
Placeholder. `placeholder` is a hint text that is shown while the field is empty and disappears as soon as there is any text; it is never part of the contents (`e.get()` still returns an empty string). It is drawn in its own colors: `placeholder_color` (the text, default the muted text color) and `placeholder_fill_color` (the background of the field while the placeholder is shown; by default the normal field color). Both can be changed with `configure` and are also accepted as keys of a `theme` dictionary, and `placeholder` itself can be changed later too. It works on the Textbox, Spinbox, Combobox, and OptionMenu as well.
Example:

```
search = ctk.Entry(root, placeholder = 'Search...',
placeholder_color = '#868e96')
search.grid()
```

Control-a selects all. `e.set_text(text)` replaces the text even when the entry is readonly or disabled. See Native widgets inside for the forwarded methods.

5.7 Textbox

Use `tb = ctk.Textbox(master, {args})` to make a multi-line text box with a rounded border.
Example:

```
tb = ctk.Textbox(root, wrap = 'word', width = 50, height = 12)
tb.insert('1.0', 'Some text')
tb.tag_configure('bold', font = ('TkDefaultFont', 10, 'bold'))
tb.grid()
```

Args: `width` and `height` in characters and lines (default `80` by `24`, the same as tkinter's `Text`), `placeholder` (a hint shown while the text box is empty, with `placeholder_color` and `placeholder_fill_color`, exactly like the Entry's), `scrolled` (`True` adds built-in canvas scroll bars that hide themselves when not needed, default `False`), `orient` (with `scrolled`: `'vertical'`, the default, with the bar on the right; `'horizontal'`, with the bar at the bottom; or `'both'`), `font`, `wrap` (`'char'` by default, like tkinter's `Text`; `'word'` or `'none'`), `state`, `yscrollcommand`, and `xscrollcommand` (for scroll bars of your own; they also work together with `scrolled`), `scrollbar_width` (the width of the built-in scroll bars in pixels, default `14`), `line_numbers` (`True` shows the number of each line in a gutter at the left of the text, default `False`; see below), `line_number_font` (the font of the numbers, by default the font of the text), `line_number_justify` (`'left'`, `'center'`, or `'right'`, the default: where the numbers sit in the gutter), `radius` (default the theme key `radius`), `border_width`, `focus_border_width`, and every other option of `tkinter.Text` (for example `undo`, `padx`, `spacing1`).
Colors: the same as the Entry's, for the line numbers `line_number_color` (default the muted text color), `line_number_current_color` (the line of the cursor, default the text color), and `line_number_fill_color` (the gutter, by default the color of the text box), and for the built-in scroll bar `scrollbar_track_color`, `scrollbar_thumb_color`, and `scrollbar_thumb_hover_color`.
Line numbers. With `line_numbers = True` (or later `tb.configure(line_numbers = True)`) the numbers follow the text as it is edited, scrolled, resized, or changed by undo and redo. A line that wraps (`wrap 'word'` or `'char'`) is numbered once, at its first row, and not while that row is scrolled out of view. The gutter widens by itself when the text gets more than 999 lines. Once the line numbers have been shown, the inner `Text` generates the virtual event `<<TextboxChanged>>` whenever its text, cursor, or view changes.
Control-a selects all. Everything `tkinter.Text` has (`get`, `insert`, `delete`, `index`, `see`, `tag_*`, `mark_*`, `search`, `edit_*`, `yview`...) works directly on the textbox. Use `tb.inner` for the `Text` itself.

5.8 Spinbox

Use `s = ctk.Spinbox(master, from_, to, increment, {args})` to make a number entry with two arrows.
Example:

```
s = ctk.Spinbox(root, from_ = 0, to = 10, increment = 0.5)
s.grid()
```

Args: `from_` (default `0`), `to` (default `100`), `increment` (default `1`), `values` (a list of strings to go through instead of numbers), `wrap` (go from the last value back to the first), `command` (called with no arguments after an arrow or the Up/Down keys changed the value), `width` (default `8`), and all the args of the Entry (including all its colors). The color `arrow_active_color` is used for an arrow while it is pressed.
Only text that can be the start of a valid value can be typed or pasted: digits (with a minus sign if the range goes below zero, and a decimal point with as many decimals as `increment` and the range use), or, with `values`, the beginning of one of the values. Everything else is rejected as you type. Pass your own `validatecommand` to replace this rule. Holding an arrow repeats. The Up and Down keys step too. When the entry loses the focus, the value is corrected to a number inside the range (or to one of `values`). Method: `s.step(1)` and `s.step(-1)`.

5.9 Combobox

Use `cb = ctk.Combobox(master, values, {args})` to make an entry with a drop-down list of values.
Example:

```
cb = ctk.Combobox(root, values = ['Python', 'LaTeX', 'Plain text'],
state = 'readonly')
cb.set('Python')
cb.bind('<<ComboboxSelected>>', lambda e: print(cb.get()))
cb.grid()
```

Args: `values`, `command` (called with the picked value), `postcommand` (called, with no arguments, right before the list opens, so it can update `values`), `state` (`'readonly'` makes the whole field open the list instead of being editable), `default` (the text the combo box starts with; it does not have to be one of the values, and `command` is not called for it; it also works when the combo box is `'readonly'` or `'disabled'`; `cb.set(text)` and `cb.current(index)` do the same later), `popup_font`, `popup_max_height`, and all the args of the Entry.
The drop-down list does not grow to the bottom of the screen: it is at most `popup_max_height` pixels tall (default `260`), and a longer list scrolls. It has a scroll bar that you can drag (click the track to page), and you can also use the mouse wheel and the Up, Down, Home, and End keys. The list opens scrolled so that the current value is in view.

`cb.configure(popup_max_height = 150)` changes the limit later. The width of its scroll bar is `popup_scrollbar_width` (default `12`), also for the OptionMenu; a long Menu has `menu.configure(scrollbar_width = ...)`.
Methods: `cb.set(value)`, `cb.get()`, `cb.current()` (the index of the value, or `-1`) and `cb.current(index)` (sets it). The Down key opens the list.
Colors: all the Entry's, and for the drop-down list: `popup_fill_color`, `popup_border_color`, `popup_hover_color` (the hovered row), `popup_hover_text_color`, `popup_text_color`, `popup_disabled_text_color`, `popup_muted_text_color`, `popup_current_color` (the row of the current value), `popup_separator_color`, `popup_thumb_color` (the scroll indicator).

5.10 OptionMenu

Use `o = ctk.OptionMenu(master, variable, default, *values, {args})` to make a drop-down button for choosing one of some values. The values can also be given as a list with `values`: `ctk.OptionMenu(root, None, 'Light', values = ['Light', 'Dark'])`.
Example:

```
v = tkinter.StringVar()
o = ctk.OptionMenu(root, v, 'Light', 'Light', 'Dark',
command = lambda value: print('chose', value))
o.grid()
```

The `default` is selected at the start and added to the values if it is missing. Pass `None` as the variable to let the widget make its own one (`o.variable`).
Args: `solid` (`True` fills the button with the accent color like a filled Button: it uses the same theme keys as the Button: `accent` for the fill, `accent_hover` while hovered, `accent_press` while the list is open, `accent_text` for the text and the arrow, `button_border` for the border, and `focus_ring` while focused or open; default `False`; it can be changed later with `configure`, and the color options below still win when given; the corners are those of any dropdown (`radius`), and a disabled solid button has the disabled look of a Button: `surface_disabled` for the fill, `text_disabled` for the text and the arrow, and `border` for the border), `command` (called with the picked value), `placeholder` (text shown while nothing is chosen, that is while the variable is empty, in `placeholder_color`, with `placeholder_fill_color` as the background; the same options as the Entry's), `radius`, `border_width`, `focus_border_width`, `popup_font`, `popup_max_height`, `popup_scrollbar_width`, `font`, `state`.
The list is at most `popup_max_height` pixels tall (default `260`); a longer list scrolls, exactly like the Combobox's (draggable scroll bar, mouse wheel, Up/Down/Home/End, opening at the current value). The `values` list can be changed later: `o.configure(values = [...])`.
Colors: `fill_color`, `hover_fill_color`, `disabled_fill_color`, `border_color`, `hover_border_color`, `focus_color`, `disabled_border_color`, `text_color`, `disabled_text_color`, `arrow_color`, `disabled_arrow_color`, and all the drop-down list colors (`popup_*`) of the Combobox.

5.11 Menu

Use `m = ctk.Menu(master, {args})` to make a popup menu. It is not a widget, it is an object that opens a popup (with submenus) where you tell it to.
Example:

```
m = ctk.Menu(root)
m.add_command(label = 'New', accelerator = 'Ctrl+N',
command = lambda: print('new'))
m.add_command(label = 'Disabled', state = 'disabled')
m.add_separator()
recent = ctk.Menu(root)
recent.add_command(label = 'notes.txt', command = lambda: print('notes'))
m.add_cascade(label = 'Recent', menu = recent)
wrap = tkinter.BooleanVar(value = True)
m.add_checkbutton(label = 'Word wrap', variable = wrap)
root.bind('<Button-3>', lambda e: m.post(e.x_root, e.y_root))
```

Adding entries (the `insert_*` versions take the position as their first argument):

- `add_command(label, command, accelerator, state)`
- `add_separator()`
- `add_checkbutton(label, variable, onvalue, offvalue, command, accelerator, state)`
- `add_radiobutton(label, variable, value, command, accelerator, state)`
- `add_cascade(label, menu, state)`

Other methods: `m.post(x, y)` (or `tk_popup`) shows the menu at screen coordinates, `m.unpost()`, `m.delete(first, last)`, `m.index(label_or_position)`, `m.entryconfigure(index, **options)`, `m.entrycget(index, option)`, and `m.invoke(index)`. Positions can be numbers, `'end'`, or a label.
The popup closes when you click outside it, press Escape, move the window, or pick an entry. You can use the mouse, or Up, Down, Home, End, Return, Right (open a submenu), Left, and Escape.
Colors (arguments of `Menu`, or `m.configure(...)`): `fill_color`, `border_color`, `hover_color`, `hover_text_color`, `text_color`, `disabled_text_color`, `muted_text_color` (the accelerators), `current_color`, `separator_color`, `thumb_color`. The `font` is an argument too.
Note: The popup is a borderless window, so its corners are square.

5.12 MenuBar

Use `mb = ctk.MenuBar(master, menu)` to make a bar like a window's menu bar. It shows the cascade entries of the Menu you give it.
Example:

```
bar = ctk.Menu(root)
file_menu = ctk.Menu(root)
file_menu.add_command(label = 'Quit', command = root.destroy)
bar.add_cascade(label = 'File', menu = file_menu)
ctk.MenuBar(root, bar).pack(fill = 'x')
```

Clicking a name opens its menu below it; while a menu is open, hovering another name switches to it. The bar updates itself when you add or remove cascades from the menu. Use `mb.set_menu(menu)` to show another menu.
Colors: `hover_color` (the fill of the name under the pointer; by default the hover variant of the color the bar sits on, so it is darker on a light bar and lighter on a dark one), `open_color` (the fill of the name whose menu is open, the accent hover color by default), `open_text_color` (its text, `accent_text` by default), `text_color`, `disabled_text_color`.

5.13 Scale

Use `s = ctk.Scale(master, from_, to, {args})` to make a slider.
Example:

```
s = ctk.Scale(root, from_ = 0, to = 100, length = 300,
command = lambda v: print(v))
s.set(35)
s.grid()
```

Args: `from_`, `to`, `orient` (`'horizontal'` or `'vertical'`), `variable` (a `DoubleVar` is made if not given), `command` (called with the new value as a number), `resolution` (the step; `0` means any value), `length` (in pixels, default `160`), `value` (the starting value), `track_thickness` (default `6`), `thumb_size` (default `18`).
Methods: `s.get()` and `s.set(value)`. You can click or drag, use the mouse wheel, the arrow keys, Home, and End.
Colors: `track_color`, `progress_color` (the filled part), `disabled_progress_color`, `thumb_color`, `thumb_hover_color`, `disabled_thumb_color`, `thumb_border_color`, `thumb_hover_border_color`, `disabled_thumb_border_color`, `focus_color`.

5.14 Progressbar

Use `p = ctk.Progressbar(master, {args})` to make a progress bar.
Example:

```
p = ctk.Progressbar(root, length = 300)
p.configure(value = 40)
p.grid()
busy = ctk.Progressbar(root, length = 300, mode = 'indeterminate')
busy.grid()
busy.start()
```

Args: `orient`, `length` (default 200), `mode` (`'determinate'` or `'indeterminate'`), `maximum` (default 100), `value`, `variable`, `track_thickness` (default 10). Colors: `track_color` and `progress_color`.
Methods: `p.start(interval = 15)` switches to the indeterminate mode and animates a bouncing bar, `p.stop()`, and `p.step(amount = 1)` adds to the value (and wraps around the maximum).

5.15 Meter

Use `m = ctk.Meter(master, {args})` to make a round meter: a ring that fills up to a value, with the value written in the middle.
Example:

```
v = tkinter.DoubleVar(value = 65)
ctk.Meter(root, variable = v, suffix = '%', subtext = 'drag me',
interactive = True).grid()
ctk.Meter(root, value = 30, shape = 'semi', subtext = 'semi').grid()
```

The `shape` is `'gauge'` (a ring with a gap at the bottom, the default), `'circle'` (a whole ring that starts at the top), or `'semi'` (half a ring). The value goes from `from_` (default 0) to `to` (default 100); read or assign it with `m.value` or let a tk `variable` hold it. The text in the middle is the value followed by `suffix`, unless you give `text`, and `subtext` is a smaller line under it. The value in the text is rounded to `decimals` places (by default as many as `step` has, so none for a step of 1), so a value that is set from outside, for example by a slider, never writes out all its decimals; the value itself is not changed, and `m.shown_value()` returns the rounded text.
With `interactive` set to `True` the ring can be dragged (and the meter takes the keyboard focus, so the arrow keys step it), and `command(value)` is called after each change; `step` (default 1) is what the value is rounded to and moved by.
Args: `value`, `from_`, `to`, `variable`, `size` (the width in pixels, default 180; the height follows from the shape), `thickness` (the width of the ring, default 16), `shape`, `text`, `suffix`, `subtext`, `interactive`, `command`, `step`, `decimals`. Colors: `track_color`, `indicator_color` (the filled part), `text_color`, and `subtext_color`.
Method: `m.fraction()` returns how far the ring is filled, from 0 to 1.

5.16 ColorDisplay

Use `c = ctk.ColorDisplay(master, {args})` to show one color as a small rounded square, like the swatches of the color picker.
Example:

```
c = ctk.ColorDisplay(root, color = '#ff8800')
c.grid()
c.config(color = '#3366cc')
```

Args: `color` (a tk color such as `'#ff8800'` or `'red'`, or a theme key; default `'#000000'`), `size` (the side in pixels, default 24; `width` and `height` override it), `radius` (the corner radius; the default is the theme key `small_radius`, 6), `border_width` (default 1), and `highlight` (`None`, `'hover'`, or `'selected'`: a thicker border in the theme's text or accent color). Color: `border_color` (the border follows the theme by default).

5.17 Scrollbar

Use `sb = ctk.Scrollbar(master, command, {args})` to make a scroll bar. It connects to widgets exactly like the tkinter one.
Example:

```
lb = ctk.Listbox(root, ['Item %d' % n for n in range(100)])
sb = ctk.Scrollbar(root, command = lb.yview)
lb.configure(yscrollcommand = sb.set)
lb.grid(row = 0, column = 0)
sb.grid(row = 0, column = 1, sticky = 'ns')
```

Args: `orient` (`'vertical'` or `'horizontal'`), `command`, `autohide` (draw nothing while everything is visible, default `False`), `thickness` (the width in pixels, default 14), `radius` (the corner radius of the track and the thumb; the default is the theme key `scrollbar_radius`, which is large enough to make them fully round). You can drag the thumb, click the track to page, and use the mouse wheel on it. Methods: `sb.set(first, last)` and `sb.get()`. Colors: `track_color`, `thumb_color`, `thumb_hover_color`, `disabled_thumb_color`.

5.18 Separator

Use `ctk.Separator(master, orient, thickness)` to draw a line (`fill = 'x'` for a horizontal one, `fill = 'y'` for a vertical one).
Example:

```
ctk.Separator(root).pack(fill = 'x', padx = 10, pady = 8)
```

Args: `orient` (default `'horizontal'`), `thickness` (default 1), and the color `color`.

5.19 Listbox

Use `lb = ctk.Listbox(master, contents, {args})` to make a list that you can select from. `contents` is a list (or tuple) of the items.
Example:

```
lb = ctk.Listbox(root, ['One', 'Two', 'Three'], height = 5,
selectmode = 'extended')
lb.bind('<<ListboxSelect>>', lambda e: print(lb.curselection()))
lb.grid()
```

Args: `width` (characters, default `20`), `height` (rows, default `10`), `selectmode` (`'browse'`, `'single'`, `'multiple'`, or `'extended'`; extended supports Ctrl-click, Shift-click, and dragging), `yscrollcommand`, `scrolled` (`True` draws a scroll bar inside the box, on the right, that only shows while the list is longer than the box; you can drag the thumb and click the track to page; default `False`), `font`, `radius` (default the theme key `radius`), `border_width`, `focus_border_width`, `scrollbar_width` (the width of the built-in scroll bar in pixels, default `10`), `row_padding` (extra pixels of height per row, default `8`), `row_radius` (the corner radius of the selection highlight, default `5`).
With `scrolled` you do not need a separate Scrollbar; `yscrollcommand` still works if you also want to connect a scroll bar of your own.
The methods are the ones of the tkinter `Listbox`: `insert`, `delete`, `get`, `size`, `index`, `curselection`, `selection_get` (the text of the selected rows, one per line), `selection_set`, `selection_clear`, `selection_includes`, `selection_anchor`, `activate`, `nearest`, `see`, `yview`, `yview_moveto`, `yview_scroll`. The keys Up, Down, Page Up, Page Down, Home, End, and the mouse wheel work. Double clicks can be bound normally with `<Double-Button-1>`.
Colors: `fill_color`, `disabled_fill_color`, `border_color`, `focus_color`, `disabled_border_color`, `text_color`, `disabled_text_color`, `hover_color`, `selected_color` (the selected rows while the list has the focus), `selected_inactive_color`, `selected_text_color`, and for the built-in scroll bar `scrollbar_track_color`, `scrollbar_thumb_color`, and `scrollbar_thumb_hover_color`.

5.20 Notebook

Use `nb = ctk.Notebook(master, {args})` to make tabs. Make each page with the notebook as its master, then add it with `add`.
Example:

```
nb = ctk.Notebook(root)
nb.pack(fill = 'both', expand = True, padx = 10, pady = 10)
page1 = ctk.Frame(nb)
ctk.Label(page1, 'Page one').pack(padx = 20, pady = 20)
nb.add(page1, text = 'First')
page2 = ctk.Frame(nb)
ctk.Button(page2, 'Page two').pack(padx = 20, pady = 20)
nb.add(page2, text = 'Second')
nb.bind('<<NotebookTabChanged>>', lambda e: print(nb.index('current')))
```

The notebook sizes itself to its biggest page, and the pages of a `Frame` take the page background color automatically.
Styles. `style = 'segmented'` (the default) draws a rounded segmented control (like macOS or customtkinter's tab view) above a rounded page, with the selected tab filled with the accent color. `style = 'classic'` draws folder-like tabs joined to the page:

```
nb = ctk.Notebook(root, style = 'classic')
```

Closable tabs. `closable = True` puts a close button on every tab. A single tab can override it with its own `closable`:

```
nb = ctk.Notebook(root, closable = True,
on_close = lambda page: print('closing', page))
nb.add(page1, text = 'First', closable = False)
```

Clicking the × (or middle-clicking the tab) calls `on_close(page)`; if it returns `False` the tab stays. Otherwise the tab is removed, and its page is destroyed unless you pass `destroy_on_close = False`. Then `<<NotebookTabClosed>>` is generated.

Draggable tabs. `draggable = True` lets the user reorder the tabs by dragging them along the strip (press a tab, move the mouse a few pixels, and drag; the cursor changes while dragging). The tabs swap places live as the dragged tab passes the middle of its neighbor, the selected tab stays selected, and when the strip overflows, dragging near its ends scrolls it. `<<NotebookTabMoved>>` is generated when the tab is dropped. It works in both styles, together with closable tabs.

Many tabs. When the tabs do not fit, an arrow appears at each end of the strip (`<` on the left, `>` on the right) (they highlight when hovered and dim at the ends); each click brings the next hidden tab into view. The mouse wheel over the strip scrolls it too.

Args: `font`, `radius` (the corner radius of the page, and of the tabs in the classic style; the default is the theme key `radius`, `8`), `segment_radius` (the corner radius of the tabs in the segmented style; the default is the theme key `tab_radius`, `10`, round corners; a smaller number such as `6` gives softer corners, `0` is square, and a big number such as `99` makes fully rounded pills; `None` is the theme's value; the pill behind the tabs follows it), `padding` (space around the page, default `8`), `tab_padx` (default `16`), `closable`, `on_close`, `destroy_on_close`, `draggable` (default `False`), `style`. All of them, including `style`, can be changed later with `configure` (`nb.configure(style = 'classic')` redraws the tabs and moves the page to the new look at once).

Methods (tabs can be given as a page widget, a number, `'current'`, or `'end'`):

- `nb.add(page, text, state, image, closable)` and `nb.insert(where, page, ...)`. `state` is `'normal'`, `'disabled'`, or `'hidden'`.
- `nb.forget(tab)` removes a tab without destroying the page, `nb.hide(tab)` hides it.
- `nb.move(tab, position)` moves a tab to a new position (a number or `'end'`) from your code; the selected tab stays selected.
- `nb.select()` returns the selected page, `nb.select(tab)` selects one.

- `nb.tab(tab, option)` reads and `nb.tab(tab, option = value)` changes the `text`, `state`, `image`, `closable`, or `tooltip` of a tab. A tab with a `tooltip` (a text, given to `nb.add` or `nb.insert` too) shows it in a small tooltip while the pointer rests on the tab.
- `nb.tabs()` (the pages), `nb.index(tab)`, `nb.close_tab(tab)`, `nb.is_closable(tab)`.
- `nb.enable_traversal()` makes Control-Tab and Control-Shift-Tab switch tabs. The Left and Right keys switch tabs while the notebook has the focus.
- `nb.update_size()` recalculates the requested size after you changed the contents of a page.

Colors: `tab_color`, `tab_hover_color`, `tab_text_color`, `disabled_tab_text_color`, `selected_tab_color`, `selected_tab_text_color`, `border_color`, `page_color` (the page and the background of its Frames; the window color by default), `track_color` (the pill behind the segmented tabs), `close_color`, `close_hover_color` (the × turns red when hovered), `arrow_color`, `arrow_hover_color`, `arrow_hover_text_color`.

5.21 Panedwindow

Use `pw = ctk.Panedwindow(master, orient)` to make resizable panes separated by draggable sashes (with a grip drawn on each). Make the panes with the panedwindow as their master; panes made as children of the panedwindow's own master also work, and are placed inside the panedwindow (so they never extend past it).
Example:

```
pw = ctk.Panedwindow(root, orient = 'horizontal')
pw.pack(fill = 'both', expand = True)
pw.add(ctk.Listbox(pw, ['a', 'b', 'c']), weight = 0)
right = ctk.Panedwindow(pw, orient = 'vertical')
pw.add(right, weight = 1)
right.add(ctk.Textbox(right), weight = 1)
right.add(ctk.Button(right, 'Bottom pane'), weight = 1)
```

The `weight` says how the panes share extra space when the window is resized (a pane with weight `0` keeps its size; if all weights are `0` the last pane gets the extra space). A pane starts with the size it asks for and cannot become smaller than 24 pixels.
Methods: `pw.add(pane, weight)`, `pw.insert(where, pane, weight)`, `pw.forget(pane)` (or `remove`), `pw.panes()`, `pw.paneconfigure(pane, weight = 2)` (or `pane`), `pw.sashpos(index)` reads and `pw.sashpos(index, position)` sets the position of a sash.

Draggable panes. `draggable = True` (default `False`; can be changed later with `configure`) lets the user move a pane to another place by dragging it, like the tabs of a Notebook: press the pane's *handle*, move the mouse a few pixels, and drag; the cursor changes while dragging, and the pane takes the place of the pane under the pointer, live. The sizes belong to the places, so they stay where they were. `<<PanedwindowPaneMoved>>` is generated when the pane is dropped. The `handle` of a pane (an argument of `add` and `insert`, or set later with `pw.pane_handle(pane, handle)`, which returns the handle when called without one, or `pw.paneconfigure(pane, handle = ...)`) is what picks the pane up:

- `'frame'` (the default) — the pane widget itself, but not the widgets inside it.
- `'all'` — the pane widget and everything inside it.
- a widget inside the pane (such as a title `Label`) — that widget and what is inside it.
- a list of any of these, such as `['frame', title, toolbar]` — every one of them is a handle.

Example:

```
pw = ctk.Panedwindow(root, draggable = True)
title = ctk.Label(pane, 'Drag me')
pw.add(pane, handle = title)
```

`pw.move(pane, position)` moves a pane to a number or `'end'` from code (the sizes stay with the places, too).
Colors: `sash_color`, `sash_active_color` (hovered or dragged), `grip_color`, `grip_active_color`.

5.22 GridPanedwindow

Use `gp = ctk.GridPanedwindow(master)` to make resizable panes that can be split in *both* directions inside one widget. A Panedwindow only goes one way; to split one of its panes the other way you have to nest a second panedwindow in it, and every nested panedwindow is a separate widget. In a `GridPanedwindow` you take any pane, however deep, and split it horizontally or vertically, and then split the new panes again.
Example:

```
gp = ctk.GridPanedwindow(root)
gp.pack(fill = 'both', expand = True)

files = ctk.Listbox(gp, ['main.py', 'util.py'], scrolled = True)
gp.add(files)                                  # the first pane
```

```
editor = ctk.Textbox(gp)
gp.split(files, editor, 'horizontal')      # files | editor

output = ctk.Textbox(gp)
gp.split(editor, output, 'vertical')       # editor above output

terminal = ctk.Textbox(gp)
gp.split(output, terminal, 'horizontal')   # output | terminal
```

This makes a list on the left, with a text box over two side-by-side text boxes on the right. The last line splits the *bottom* pane sideways, which is exactly what one panedwindow cannot do.

How it works. The panes form a tree of rows and columns. Every row or column (a *split*) has its own sashes between its panes, and dragging a sash moves only the border between its two neighbors. The panes are the widgets you give it, created with the `GridPanedwindow` as their master (also the ones deep in the layout); the widget places them itself, so do not `pack` or `grid` them. Each pane keeps its proportion of the space when the window is resized, and no pane can be made smaller than 24 pixels by dragging.

Junctions. Where a sash ends on a sash of the other direction (for example, a horizontal sash in the right column meeting the vertical sash between the columns), the two meet at a *junction*. Press and drag near that point, within about 9 pixels, and every sash that meets there moves together: the vertical one sideways and the horizontal one up or down at the same time. The pointer becomes a four-way move cursor and all the sashes of the junction light up while you are near it. If the panes on both sides have sashes that line up, they all move together. Dragging a sash anywhere else along its length moves only that sash, as before.

Methods:

- `gp.add(pane)` adds the first pane. Later, `gp.add(pane, orient)` appends a pane to the whole layout: to the right of everything (`'horizontal'`, the default) or below everything (`'vertical'`), sharing the space equally with the other top-level panes.
- `gp.split(target, pane, orient, before = False, ratio = 0.5)` splits the pane `target` (a widget that is already in the layout) in two: `pane` goes to its right (`'horizontal'`) or below it (`'vertical'`), or to its left or above it with `before = True`. `ratio` is the part of the target's space that the new pane gets. Only the target's own space is divided.
- `gp.remove(pane)` (or `forget`) takes a pane out of the layout and gives its space to its neighbors; the widget is not destroyed. A row or column left with one pane disappears.
- `gp.panes()` returns all the panes in layout order.
- `gp.swap(pane, other)` exchanges the places of two panes (the sizes stay with the places).

Draggable panes. `draggable = True` (default `False`) lets the user pick a pane up by its handle (`'frame'`, `'all'`, a widget inside the pane, or a list of them, set with the `handle` argument of `add` and `split`, or later with `gp.pane_handle(pane, handle)`) and put it somewhere else in the layout, the way editor groups work in VS Code. While dragging, an outline shows where the pane would go, and it is moved when the mouse is released:

- over the *middle* of another pane — the two panes swap places (the sizes stay with the places);
- near a *side* of another pane (the outer 30% of its width or height) — the pane goes beside that pane on that side, splitting the pane's space in half, even when that makes a new row or column (so a pane in a row can be moved to be above or below another one);
- within about 18 pixels of the *border of the whole widget* — the pane docks along that whole side, taking the full width or height of the layout. For example, dragging `C` from `A | (B over C)` to the bottom border makes `(A | B)` over `C`.

The place the pane left closes up (a row or column left with one pane disappears, and rows that become the same direction as the row around them merge into it). `<<PanedwindowPaneMoved>>` is generated when a pane was moved. **Example:**

```
terminal_pane = gp.panes()[-1]
gp.remove(terminal_pane)
terminal_pane.destroy()
```

Colors: `sash_color`, `sash_active_color` (hovered or dragged), `grip_color`, `grip_active_color`, like the Panedwindow.

5.23 Frame

Use `f = ctk.Frame(master, {args})` to make a frame. It is a normal `tkinter.Frame` that follows the appearance mode: it has the same background as its master (a notebook page, a window...).
Example:

```
f = ctk.Frame(root)
f.pack(padx = 10, pady = 10)
ctk.Label(f, 'Inside a frame').pack()
```

Give `bg` for a fixed color that does not follow the theme. A frame can have its own theme and appearance mode for everything inside it: `theme`, `appearance_mode`, `f.set_theme(...)`, and `f.set_appearance_mode(...)` (see Changing the theme dynamically). All other args are the same as tkinter's.

5.24 ScrolledFrame

Use `sf = ctk.ScrolledFrame(master, {args})` to make a frame that scrolls. Put your widgets inside the ScrolledFrame itself; `pack`, `grid`, and `place` act on the whole scrolling container.
Example:

```
sf = ctk.ScrolledFrame(root, orient = 'vertical')
sf.pack(fill = 'both', expand = True)
for n in range(50):
    ctk.Checkbutton(sf, 'Option %d' % n).pack(anchor = 'w')
```

Args: `orient` (`'vertical'` (default), `'horizontal'`, or `'both'`), `autohide` (the scroll bars hide themselves when not needed, default `True`), `scrollbar_width` (the width of the scroll bars in pixels, default `14`), `bg`, and the scroll bar colors `scrollbar_track_color`, `scrollbar_thumb_color`, `scrollbar_thumb_hover_color`.
The contents keep the size they ask for along the directions that scroll (so they never get cut off when widgets are added later), and are as big as the visible area along the others. The mouse wheel scrolls vertically (horizontally for `'horizontal'`), and Shift with the wheel always scrolls horizontally. Text boxes and list boxes inside keep their own wheel scrolling. Scrolled areas can be nested (a scrolled Foldable or LabelFrame inside a ScrolledFrame, for example): the wheel scrolls the innermost one under the pointer, and once that one is at its end, the one around it.
Example:

```
wide = ctk.ScrolledFrame(root, orient = 'horizontal')
wide.pack(fill = 'x')
for n in range(30):
    ctk.Button(wide, 'Button %d' % n).pack(side = 'left', padx = 6, pady = 10)
```

5.25 Foldable

Use `f = ctk.Foldable(master, text, {args})` to make a section with a header bar (an arrow and a title) that folds its contents away when you click it. Put your widgets inside the Foldable itself, exactly as in a Frame; `pack`, `grid`, and `place` act on the whole section.
Example:

```
f = ctk.Foldable(root, 'Details')
ctk.Label(f, 'Everything in here is an ordinary child.').pack(anchor = 'w')
ctk.Checkbutton(f, 'A check box').pack(anchor = 'w')
f.pack(fill = 'x')
ctk.Label(root, 'Stays where it is.').pack()
```

A Foldable is one self-contained widget: the header and the contents live in a container of their own, and the parent only ever sees that one container. Folding changes the size of the Foldable and nothing else, so it never adds widgets to

the parent or moves the parent's other widgets around, apart from the space the Foldable itself takes. It can be nested inside another Foldable, and the contents are folded away with `grid_remove`, so they keep their own layout and values.
Args: `text` (the title), `expanded` (whether the contents start open, default `True`), `command(expanded)` (called after every fold or unfold), `padding` (the space around the contents, default `8`), `scrolled`, `orient`, `max_height`, and `max_width` (see below), `state` (`'normal'` or `'disabled'`), `font` (the font of the title, bold by default; also changed with `f.configure(font = ...)`), `bg`, `theme`. The header takes the colors `fill_color`, `hover_color`, `text_color`, `arrow_color`, `focus_color`, and `disabled_text_color`.
A disabled Foldable draws its header greyed out and cannot be folded or unfolded by clicking it or with the keyboard; `f.expand()`, `f.collapse()`, and `f.toggle()` from code still work, and the widgets inside are not affected. Change it with `f.configure(state = 'disabled')`. The title can be changed with `f.configure(text = ...)` as well as `f.text`.
With `scrolled = True` the contents scroll when they are bigger than `max_height` pixels tall (default `300`) or, with an `orient` that scrolls sideways, `max_width` pixels wide (default `300`); smaller contents take only the room they need. `orient` is `'vertical'` (the default), `'horizontal'`, or `'both'`, as for a ScrolledFrame. The scroll bars show when they are needed and the mouse wheel works over the contents. The children are still added to the Foldable itself, and the scroll bar colors are `scrollbar_track_color`, `scrollbar_thumb_color`, and `scrollbar_thumb_hover_color`, as for a ScrolledFrame.
Methods: `f.expand()`, `f.collapse()`, `f.toggle()`, and `f.is_expanded()`; the title is `f.text` and can be assigned. Clicking the header, or pressing Space or Return on it, folds or unfolds. Every fold or unfold generates the virtual event `<<FoldableToggled>>` on the Foldable.

5.26 LabelFrame

Use `lf = ctk.LabelFrame(master, text, {args})` to make a frame with a rounded outline and a title set into the top edge of the outline. Put your widgets inside the LabelFrame itself, exactly as in a Frame; `pack`, `grid`, and `place` act on the whole thing.
Example:

```
lf = ctk.LabelFrame(root, 'Options')
ctk.Checkbutton(lf, 'Remember me').pack(anchor = 'w')
ctk.Button(lf, 'Apply').pack(anchor = 'w')
lf.pack(fill = 'x', padx = 10, pady = 10)
```

Args: `text` (the title; assign `lf.text` to change it), `padding` (the space between the outline and the contents, default `8`), `radius` (default the theme key `radius`, `8`), `border_width` (default `1`), `scrolled`, `orient`, `max_height`, `max_width`, and `scrollbar_width` (the contents scroll exactly as in a Foldable), `font` (the font of the title; also changed with `lf.configure(font = ...)`), `bg`, `theme`. Colors:

`border_color`, `text_color`, and the scroll bar colors of a ScrolledFrame. The title can be changed with `lf.configure(text = ...)` as well as `lf.text`.

5.27 Tooltip

Use `ctk.Tooltip(widget, text, {args})` to show a popup text when the mouse rests on a widget.
Example:

```
b = ctk.Button(root, 'hover me')
b.grid()
ctk.Tooltip(b, 'A canvas-drawn tooltip')
```

Args: `delay` (in milliseconds before it shows, default `500`), `font`, `fill_color`, `border_color`, `text_color`. The text can have several lines. It hides when the mouse leaves or is pressed. It does not replace the widget's own bindings.

6 Windows

6.1 Window

`root = ctk.Window()` makes the application window. It is a `tkinter.Tk`, so everything such as `title`, `geometry`, `minsize`, `protocol`, and `mainloop` works. Its background follows the appearance mode, unless you give `bg` a color such as `'#ffffff'` (then it stays that color).
Args: `bg` and `theme`, a dictionary of theme keys that applies to the window and to every widget inside it. A window given a theme (even an empty dictionary) uses *only* that: the theme of the application (`ctk.set_theme`) does not apply to it, and the keys the dictionary does not have are the library's defaults:

```
root = ctk.Window(theme = {'window': '#14213d', 'surface': '#1d2d50',
'text': '#ffffff', 'accent': '#fca311'})
```

Methods (see Changing the theme dynamically):

- `root.set_theme(theme)` changes theme keys for the window and all the widgets in it, immediately. A value of `None` removes a change.
- `root.get_theme()` returns the window's current theme as a dictionary.
- `root.reset_theme()` removes every change made with `theme` or `set_theme`. A window that was given a theme is cut off from the application's theme (see below), so it falls back to the library's defaults, not to the application's theme.
- `root.reset_full_theme()` resets the window completely to the library's defaults: its theme is removed, its appearance mode is the default one (`'light'`), and it has no connection with the theme or the appearance mode of the application any more.

6.2 Toplevel

`top = ctk.Toplevel(root)` makes a subwindow, with the same arguments and methods. By default it is a real transient subwindow of its master (`transient = True`): the window manager keeps it grouped with the main window (a single taskbar entry, minimized and raised together with it, and shown above it). Pass `transient = False` for a completely independent window. Its theme is independent of the main window's:

```
top = ctk.Toplevel(root, theme = ctk.CANVAS_DARK_PALETTE)    # a dark window
```

7 File dialogs

`canvastk.filedialog` is a drop-in replacement for `tkinter.filedialog`: the same functions, classes, options, and return values, but drawn with canvastk widgets, so it matches your application (and follows the theme of the window it belongs to) instead of the system dialog.
Example:

```
from canvastk import filedialog

path = filedialog.askopenfilename(
    title = 'Open a text file',
    filetypes = [('Text files', '*.txt'), ('Python files', '*.py *.pyw'),
                 ('All files', '*')])
if path:
    print('opening', path)

paths = filedialog.askopenfilenames()                  # a tuple of paths
target = filedialog.asksaveasfilename(defaultextension = '.txt')
folder = filedialog.askdirectory(mustexist = True)
with filedialog.asksaveasfile(mode = 'w') as handle:  # an open file, or None
    handle.write('hello')
```

7.1 Functions

- `askopenfilename(**options)`: choose an existing file; returns its path, or `''` if cancelled.
- `askopenfilenames(**options)`: choose several files; returns a tuple of paths (empty if cancelled).
- `asksaveasfilename(**options)`: choose where to save; returns the path, or `''` if cancelled.
- `askdirectory(**options)`: choose a folder; returns its path, or `''` if cancelled.

- `askopenfile(mode = 'r', **options)`, `askopenfiles(mode = 'r', **options)`, and `asksaveasfile(mode = 'w', **options)`: the same, but they return the file(s) opened in `mode` (a list for `askopenfiles`), or `None` if cancelled.
- The classes `Open`, `SaveAs`, and `Directory`: `Open(master, **options).show(**more_options)` does what the matching function does. Options given to `show` override the ones given to the constructor.

7.2 Options

- `parent`: the window the dialog belongs to (the default window if omitted). The dialog is modal, centered on it, and grouped with it by the window manager.
- `title`: the dialog's title (`'Open'`, `'Save As'`, or `'Select Folder'` by default).
- `initialdir`: the folder to start in. By default the folder the previous file dialog ended in, or the current folder.
- `initialfile`: the file name to fill in (it may include a folder, which then becomes the starting folder; in an open dialog the file is also selected).
- `filetypes`: a list of `(label, patterns)`. `patterns` is a string such as `'*.png *.jpg'` or a sequence of patterns; a pattern may be a glob (`'*.txt'`), an extension (`'.txt'` or `'txt'`), or `'*'` for everything. The first entry is selected at the start. Without `filetypes` there is a single `'All files'` entry.
- `typevariable`: a `StringVar` that gives the file type to start with (by its label) and receives the label of the type that was selected when the dialog is accepted.
- `defaultextension`: added to the name typed in a save dialog if it has no extension.
- `multiple`: allow choosing several files (`askopenfilenames` sets it).
- `mustexist`: for `askdirectory`, whether the folder must already exist (a typed name that does not exist is refused).
- `confirmoverwrite`: in a save dialog, ask before replacing an existing file (default `True`).

An unknown option raises a `TypeError`, like an unexpected keyword argument. The `message` option of `askdirectory` is accepted and ignored.

7.3 Using the dialog

- **Places** on the left jump to your home folder, its usual folders (Desktop, Documents, Downloads...), mounted drives, and the file system root.

- The **back**, **forward**, and **up** buttons move through the folders you visited. The **path bar** shows the folder as a row of clickable pieces: click any folder in it to go there (an easy way out of where you are; a long path collapses its first folders into a "..." piece). Click the empty space after the last piece, or press Control-L, to type a path instead and press Return (a path to a file opens its folder and selects the file); Escape goes back to the bar.

- The file list shows an icon, **Name**, **Size**, and **Modified**. Click a column title to sort by it, and click it again to reverse the order; folders always come first. Select with the mouse or the arrow keys (Page Up, Page Down, Home, End); Ctrl-click and Shift-click select several files when `multiple` is set. Typing letters jumps to the first name that starts with them. Double click opens a folder or accepts a file; Return accepts.

- The **Name** field holds the selected name (or the names, quoted if they contain spaces); in an open dialog you can also type a name or a path, and in a save dialog the name of the new file. While you type there, the list is narrowed to the names that contain the text, and the rest of a name that begins with the text is suggested as selected text after it. Tab or Return accepts the suggestion (Return with no suggestion showing accepts the dialog as usual), Escape hides it until you type again, and typing on replaces it. When several names fit what you typed, a dropdown lists them under the field: Up and Down move through it, a click or Return picks the highlighted name, and Escape hides it. Picking a file only puts its name in the field (so you can still change it before accepting); picking a folder enters it. If you type a path (such as `Dir/file`, `/file`, or `~/file`), the list is not narrowed and the last piece is suggested from the files and folders of that folder. The path field suggests folder names the same way (Tab or Return accepts, Escape hides; Return with no suggestion opens the path). The **Type** box filters the files by the `filetypes` (folders are always shown). **Hidden** shows hidden files.

- In save and folder dialogs (not in open dialogs), **New folder** (or the right click menu) makes a folder in the current one.

- Keyboard shortcuts: Escape cancels, Alt-Left, Alt-Right, and Alt-Up go back, forward, and up, F5 refreshes, Control-h shows or hides hidden files, and Control-l edits the path.

- The open dialog refuses names that are not files, the save dialog asks before replacing a file, and a folder that cannot be read shows an error instead of closing the dialog.

7.4 Helper functions

The logic that does not need a window is available as plain functions (useful for your own file pickers and for testing): `parse_filetypes(filetypes)`, `name_matches(name, patterns)`, `list_directory(directory, show_hidden, patterns, directories_only)` (returns the entries and an error), `sort_entries(entries, column, descending)`, `default_places()`, `resolve_path(text, directory)`, `with_default_extension(name, extension)`, `human_size(size)`, `format_modified(timestamp)`, `filter_entries(entries, text)`, `complete_name(names, text)`, and `complete_path(text, directory, show_hidden, directories_only,`

8 Message boxes

`canvastk.messagebox` is a drop-in replacement for `tkinter.messagebox`: the same functions, constants, options, and return values, drawn with canvastk widgets. A message box is modal, follows the theme of the window it belongs to, and is centered over it. The file dialogs use it for their errors and questions.
Example:

```
from canvastk import messagebox

messagebox.showinfo('Saved', 'The file was saved.')
messagebox.showerror('Error', 'Could not open the file.', detail = 'Permission denied.')
if messagebox.askyesno('Quit', 'Quit without saving?'):
    root.destroy()
answer = messagebox.askyesnocancel('Save', 'Save the changes?')  # True, False, or None
```

`showinfo`, `showwarning`, `showerror`	return `'ok'`
`askquestion`	returns `'yes'` or `'no'`
`askokcancel`, `askyesno`, `askretrycancel`	return `True` (OK, Yes, Retry) or `False`
`askyesnocancel`	returns `True`, `False`, or `None` for Cancel

All of them take `(title, message, **options)`. The options are `parent` (the window it is shown over), `default` (the button that Return presses, such as `'no'`), `detail` (a second, smaller text under the message), `buttons` (your own button labels), `icon` (`messagebox.ERROR`, `INFO`, `QUESTION`, or `WARNING`), and `type` (`OK`, `OKCANCEL`, `YESNO`, `YESNOCANCEL`, `RETRYCANCEL`, or `ABORTRETRYIGNORE`). A list of your own buttons can be given with `buttons`, as in `messagebox.askquestion('Delete', 'Delete the file?', buttons = ['Delete', 'Keep'], default =` the buttons are shown in that order, and the function returns the label of the pressed button as a string (the `ask` functions that normally return a boolean return the label too, since your labels have no yes or no meaning). `default` is then one of your labels, and closing the window (or pressing Escape) returns `None`. An unknown option raises a `TypeError`. Closing the window or pressing Escape answers Cancel, or No when there is no Cancel button.

9 Color picker

`canvastk.colorchooser` is a drop-in replacement for `tkinter.colorchooser`, drawn with canvastk widgets: the same `askcolor` function and `Chooser` class, the same options, and the same return value. The dialog is modal, follows the theme of the window it belongs to, and is centered over it.
Example:

```
from canvastk import colorchooser

rgb, hex_color = colorchooser.askcolor('#3b82f6', title = 'Pick a color')
if hex_color:                    # (None, None) when the dialog was cancelled
    print(rgb, hex_color)        # (59, 130, 246) #3b82f6
```

`askcolor(color = None, **options)` returns `((r, g, b), '\#rrggbb')` with the three numbers from `0` to `255` and a lowercase hex code, or `(None, None)` if the dialog was cancelled or closed. `colorchooser.Chooser(master, **options).show()` does the same as a class. The color to start from is the first argument or the `initialcolor` (or `color`) option, and can be a hex code (`'#3b82f6'` or `'#38f'`), a string of three numbers (`'59, 130, 246'`), a tuple, or a Tk color name; the default is white. The other options are `parent` (the window the dialog is shown over; the default is the main window) and `title`. An unknown option raises a `TypeError`, and a color that cannot be understood raises a `ValueError`.

9.1 Using the dialog

- The **square** chooses the saturation (left to right) and the brightness (top to bottom) of the current hue; press and drag in it. The **rainbow bar** under it chooses the hue.
- The **Hex** field and the **R**, **G**, and **B** fields show the color as numbers, and you can type in them: the rest of the dialog follows as soon as what you typed is a valid color. The three number fields also step with the arrow buttons and the Up and Down keys.
- The **Colors** palette has twenty-four ready-made colors, and **Recent** lists the colors you chose before, newest first (twelve at most, kept until the program ends). Click a swatch to choose it; the swatch of the current color has an accent-colored ring.
- Return chooses the color (OK) and Escape cancels, like the two buttons.

9.2 Helper functions

The conversions that do not need a window are available as plain functions: `parse_color(text)` (the `(r, g, b)` of a hex code or of `'r, g, b'`, or `None`), `rgb_to_hex(rgb)`, `rgb_to_hsv(rgb)`,

`hsv_to_rgb(hue, saturation, value)` (hue, saturation, and value are numbers from `0` to `1`), `square_image(hue, width, height)`, and `hue_strip_image(width, height)` (the gradients as Pillow images).

10 Appearance and themes

10.1 Appearance modes

`ctk.set_appearance_mode(mode)` sets the mode to `'light'` or `'dark'`. Every existing widget, window, and popup recolors immediately, and the mode is used for everything made afterwards. `ctk.get_appearance_mode()` returns `'light'` or `'dark'`.
Example:

```
ctk.OptionMenu(root, None, 'Light', 'Light', 'Dark',
command = ctk.set_appearance_mode).grid()
```

A single window, container, or widget can use the other mode: `window.set_appearance_mode('dark')` makes that window and everything in it dark while the rest of the application stays as it is, and `container.set_appearance_mode('dark')` does the same for a `Frame` or any other container and everything inside it. `widget.set_appearance_mode('dark')` does it for one widget. `None` goes back to following the surroundings. `get_appearance_mode()` on a window or widget returns the mode it really uses. `Window`, `Toplevel`, `Frame`, and every canvas-drawn widget also take `appearance_mode` when created. Dialogs (message boxes, file dialogs, and the color picker) use the mode of the window they belong to. Theme keys set with `set_theme`, on the application, the window, or the widget, still win over the mode.

10.2 Accent color

The accent color is used by buttons, selected tabs, check boxes, slider fills, selections, and so on. It is a theme key like any other, so you set it with `ctk.set_theme` (see Changing the theme dynamically):

```
ctk.set_theme({'accent': '#e64980'})
```

Giving only the accent is enough: the hover color and the other keys that follow from it are derived. Unless you give `'accent_hover'` yourself, the hover color is derived from the accent: dark and medium accents are lightened (35% towards white), and very light accents are darkened (15% towards black), so the hover is always visible (see Hover_variant). The text on top of the accent color becomes black or white automatically, depending on how light the accent is. Existing widgets recolor immediately, and the accent stays when the appearance mode changes. The keys that follow from an accent (the focus ring, the selection, and the row highlight) are worked out for the appearance mode that applies, also

when the accent was set on a window or a widget, so they change with the mode too.
The dictionary `ctk.CANVAS_COLOR_THEMES` has five ready-made accents, `'blue'` (the default), `'green'`, `'red'`, `'orange'`, and `'purple'`. Each one is a theme dictionary, so you apply it with `set_theme`:

```
ctk.set_theme(ctk.CANVAS_COLOR_THEMES['green'])
```

10.3 Theme keys

The current theme is the dictionary `ctk.CANVAS_WIDGET_THEME`. It is built from the light or dark palette (`ctk.CANVAS_LIGHT_PALETTE`, `ctk.CANVAS_DARK_PALETTE`) plus the accent colors. These are its keys, and what the widgets use them for:

- `window`: the background of windows, of frames on windows, and of notebook pages.
- `surface`, `surface_hover`, `surface_disabled`: the background of entries, text boxes, list boxes, drop-downs, and unchecked boxes, in their normal, hovered, and disabled states. The hovered color is not part of the palettes: it is derived from `surface` (see below).
- `neutral`, `neutral_hover`: unselected tabs and the segmented tab pill (the hovered color is derived from `neutral`).
- `border`, `focus_ring`: normal and focused borders.
- `text`, `text_muted`, `text_disabled`: text, secondary text (arrows, accelerators), and disabled text.
- `track`, `thumb`, `thumb_hover`: slider and scroll bar tracks and thumbs (the hovered thumb is derived from `thumb`, moving away from `track`).
- `separator`: separator lines, menu separators, and sash lines.
- `tooltip`: the tooltip background.
- `button_border`: the outline of buttons.
- `danger`: the red of the tab close button when hovered.
- `accent`, `accent_hover`, `accent_press`, `accent_text`: the accent color for buttons and highlights, its hovered and pressed versions, and the text on it.
- `selection`, `selection_text`: selected text in entries and text boxes.
- `row_current`: the highlight of the current row in lists and drop-downs.

Derived hover colors. Every hover color follows from the color it belongs to, so changing a color changes its hover too: `accent_hover` from `accent`, `surface_hover` from `surface`, `neutral_hover` from `neutral`, and `thumb_hover` from `thumb`. They use hover_variant (the thumb moves away from the track instead). A hover key you give yourself in a theme always wins, and so does a hover color option of a widget. The same holds for the hovered border (`hover_border_color`, from `border_color`) and the hovered close button of a tab (`close_hover_color`, from `close_color`).
The corner radii (in pixels) are theme keys too, the same in both appearance modes (`ctk.CANVAS_RADII`). A widget's own `radius` option (or `row_radius`, `segment_radius`...) wins over them; a big number such as `99` makes corners as round as the shape allows:

- `radius` (`8`): entries, text boxes, list boxes, drop-downs, labeled frames, foldable headers, and the page of a notebook.
- `button_radius` (`15`): buttons.
- `small_radius` (`6`): small highlights such as menu bar items, the file dialog's path bar, and color swatches.
- `row_radius` (`5`): the selected or hovered row of lists, drop-downs, and menus.
- `tab_radius` (`10`): the tabs of a notebook in the segmented style.
- `checkbox_radius` (`5`): check boxes (for the default size, scaled with it).
- `track_radius` (`99`): the tracks of sliders, progress bars, and toggle switches.
- `scrollbar_radius` (`99`): the tracks and thumbs of all scroll bars, including the ones inside list boxes, text boxes, and drop-downs. The *width* of a scroll bar is not a theme key but an option: `thickness` of a Scrollbar, `scrollbar_width` of a list box, text box, scrolled frame, foldable, or labeled frame, and `popup_scrollbar_width` of a combo box or drop-down.

For example `ctk.set_theme(\{'radius': 0, 'button_radius': 0\})` makes all of these square. You can change a key for *everything* by editing the dictionary before making your widgets (`ctk.CANVAS_WIDGET_THEME['surface'] = '##fafafa'`), but it is rebuilt when you call `set_appearance_mode` or `set_theme`; for lasting changes use `set_theme`, the named options, or a `theme` dictionary on the widgets.

10.4 Changing the theme dynamically

Every color is a theme key, and each level has methods to change keys while the program is running; everything affected recolors immediately, not only the accent. All of them take a dictionary of theme keys (see Theme keys).

A value of `None` removes a change (and in a `theme` given when a widget or window is created, `None` simply means the default). If the dictionary has an `'accent'` (a `#rrggbb` color or a color name), the keys that follow from it (`accent_hover`, `accent_press`, `accent_text`, `focus_ring`, `selection`, `selection_text`, `row_current`) are derived automatically, unless you give them yourself.
From the widest to the narrowest level (a narrower level wins):

1. **The whole application:** `ctk.set_theme(theme)`, `ctk.reset_theme()`, `ctk.get_theme()`. The changes stay when the appearance mode is changed.

2. **One window:** `ctk.Window(theme = ...)` or `window.set_theme(theme)`, `window.reset_theme()`, `window.get_theme()` (also on `Toplevel`). They apply to the window and to every widget in it. A window created with a theme (any `theme =` dictionary, even an empty one) is cut off from the application's theme: it uses the library's defaults plus its own dictionary, and `reset_theme()` brings back the defaults, not the application's theme. A window created without one follows the application.

3. **One container:** the same methods, with the same rule for a container created with a theme (any dictionary, even an empty one: it and everything inside it is cut off from everything outside), on a `Frame`, `ScrolledFrame`, `Foldable`, `LabelFrame`, `Notebook`, `Panedwindow`, or `GridPanedwindow` (and `theme` when creating it). They apply to the container and to every widget inside it, so a preview area can have a theme (and an appearance mode, see below) of its own. A container inside another one wins over the outer one, and a `Toplevel` is not affected by the containers of the window it belongs to.

4. **Everything at once back to the defaults:** `x.reset\_full\_theme()` on a window, container, widget, `Menu`, or `Tooltip` (there is none on `ctk` itself) removes its theme and colors, sets its appearance mode to the default one (`'light'`), and cuts it off from the application's theme and mode and from the containers around it, whatever they have. Everything inside a window or container follows.

5. **One widget:** `widget.set_theme(theme)`, `widget.reset_theme()`, `widget.get_theme()` (and `theme` when creating it). This works on every canvastk widget. A `Menu` and a `Tooltip` have `set_theme` too; it applies the next time they are shown.

6. **One color of one widget:** the named options such as `border_color` (see Colors).

Example:

```
import canvastk as ctk
root = ctk.Window()
b = ctk.Button(root, 'Hello')
b.grid()

ctk.set_theme({'surface': '#fffbe6', 'border': '#e67700'})   # every widget
root.set_theme({'window': '#fff3bf'})      # this window and its widgets
b.set_theme({'accent': '#e67700'})         # only this button
b.set_theme({'accent': None})              # remove that change again
print(b.get_theme()['accent'])
root.reset_theme()
ctk.reset_theme()
```

Since the palettes are dictionaries of theme keys too, you can make one window dark while the rest of the application is light:

```
root.set_theme(ctk.CANVAS_DARK_PALETTE)
```

Note: `set_theme` on a widget does not recolor the widgets inside it. Use the window's `set_theme` (or the global one) to recolor a whole window.

11 Functions

These are the functions and classes that the package exports besides the widgets:

- set_appearance_mode
- get_appearance_mode
- set_theme
- get_theme
- reset_theme
- expand_theme
- hover_variant
- window_theme
- load_canvas_image
- make_font
- resolve_color
- parent_background
- rounded_box_image
- CanvasWidget

11.1 Set_appearance_mode, Get_appearance_mode

See Appearance and themes.

11.2 Set_theme, Get_theme, Reset_theme

`ctk.set_theme(theme)` changes theme keys for the whole application, `ctk.get_theme()` returns a copy of the current global theme, and `ctk.reset_theme()` removes the changes made with `set_theme`. See Changing the theme dynamically.

11.3 Expand_theme

`ctk.expand_theme(theme)` returns the dictionary with the keys that follow from its `'accent'` filled in. `set_theme` calls it for you.

11.4 Hover_variant

`ctk.hover_variant(color)` returns a hover color for any tkinter color: lighter for dark and medium colors (35% towards white), darker for very light ones (15% towards black; a color counts as very light when its brightness is above 186 out of 255). It is what derives the hover color of an accent and of a button's `default_bg`.

11.5 Window_theme

`ctk.window_theme(widget)` returns the theme dictionary of the window that contains the widget (empty if that window is not a canvastk `Window` or `Toplevel`). It is useful when writing your own widgets or popups.

11.6 Load_canvas_image

Use `ctk.load_canvas_image(path, size = None)` to load an image file as a `PhotoImage` that can be used for the `image` option of buttons, labels, and tabs. If `size` is `(width, height)` the image is resized to it with a high quality filter. Transparency is kept.

11.7 Make_font

`ctk.make_font(font = None, size = 11, weight = 'normal')` returns a `tkinter.font.Font` from a `Font` (returned as it is), a font tuple or name, or `None` (the default family at the given size and weight). Fonts made this way are shared.

11.8 Resolve_color

`ctk.resolve_color(widget, color)` converts any tkinter color (name, `#rgb`, `#rrggbb`) to `'#rrggbb'`.

11.9 Parent_background

`ctk.parent_background(widget)` returns the background color of a widget as `'#rrggbb'`; it also works for ttk widgets.

11.10 Rounded_box_image

`ctk.rounded_box_image(width, height, radius, fill, outline = None, outline_width = 0, inset` returns a cached, anti-aliased rounded box as a `PhotoImage` (transparent outside the shape). `fill` and `outline` are `'#rrggbb'` strings (`fill` may be `None`), `inset` shrinks the box on all sides, and `corners` says which of the top-left, top-right, bottom-right, and bottom-left corners are rounded.

12 Writing your own widget

`ctk.CanvasWidget` is the base class of every canvas-drawn widget. It handles the theme, the background, the colors, hover tracking, and focus tracking, and coalesces redraws (many changes in one event cycle cause one redraw). To make your own, subclass it and implement `redraw`:

```
class Badge(ctk.CanvasWidget):
    _COLOR_PARTS = ('badge_color',)

    def __init__(self, master, text = '', **kwargs):
        super().__init__(master, width = 80, height = 26, **kwargs)
        self.text = text
        self.track_interaction()    # keeps self._hovered / self._focused

    def redraw(self, width, height):
        fill = self.part('badge_color',
        'accent_hover' if self._hovered else 'accent')
        self.draw_box(0, 0, width, height, fill, radius = height / 2)
        self.draw_content((0, 0, width, height), self.color('accent_text'),
        self.text)
```

Useful members:

- `self.part(name, default_theme_key)`: the widget's override named `name` if the user set it (through the `_COLOR_PARTS` names), otherwise the theme color `default_theme_key`. `self.color(key)` reads a theme key directly.

- `self.draw_box(x1, y1, x2, y2, fill, outline, outline_width, radius)` draws a rounded box. `self.draw_content(box, color, text, image)` draws an image and text side by side. `self.draw_chevron(x, y, direction, color)` draws an arrow head.
- `self.schedule_redraw()` requests a redraw. `self.refit()` recalculates the size from `requested_size()` (return `(width, height)` from it to make a widget that sizes to its content).
- `self.call_later(milliseconds, function)` is `after` that is cancelled automatically when the widget is destroyed. `self.watch_variable(var)` redraws when a tkinter variable changes.
- `self.font`, `self.state` (`self.is_disabled()`), `self.fg`, `self.colors`.

13 Notes and limitations

- The popups of the Menu, Combobox, OptionMenu, and Tooltip are borderless windows; their contents are drawn on a canvas, but their corners are square, because a window cannot have transparent rounded corners everywhere.
- Entry, Textbox, Spinbox, and Combobox use a native `tkinter.Entry` or `tkinter.Text` for the text itself. Frame, ScrolledFrame, Panedwindow, and Notebook are tkinter frames (with canvas drawing for tabs, sashes, and scroll bars), and `Window` and `Toplevel` are native windows. Everything else is drawn entirely on a canvas.
- A Menu is not a widget and cannot be attached to a window as its native menu bar; use a MenuBar.
- Text on a canvas uses tkinter's normal text rendering; the shapes (boxes, tracks, thumbs) are anti-aliased, while thin lines like check marks and arrows are drawn with canvas lines.
- Creating hundreds of widgets takes a moment, because every widget is a Tk canvas. For long lists, use a Listbox (which draws only the visible rows) instead of one widget per item.